# MLP Project Milestone 4: Model Building and Hyperparameter Exploration

**To understand about all the milestones in detail, Please *click here***

| **Roll number** | |
|---|---|

## Submission guidelines

1. **Make your own copy with File > Make a copy, and write only in your copy.**
2. **Once you have finished writing your report, save or export the .docx document as a PDF.**
3. **Rename the PDF file using the following exact convention: [roll-number]_26t3_M[Milestone-Number].pdf**
4. **Example: If your roll number is 21f1001234 and you are submitting the PDF for Milestone 4, your file must be named exactly: 21f1001234_26t3_M4.pdf**

## Instructions

- Use the Project data to answer the questions.
- Write each answer in the "Your answer" box under the question, and paste the Python code (wherever required) you ran into the "Your code" box as text, not as a screenshot.
- Support every claim with evidence from your own analysis of the competition data: a number, a small table or a plot (Insert > Image). Summarise long outputs instead of pasting them in full.
- The points under each question describe what an answer should cover. If a point does not apply to your dataset (for example, there are no date columns), you can skip it.
- Write in your own words. Clear reasoning backed by evidence matters more than length of your answer.
- There is no single correct answer, these questions are subjective and there can be multiple valid explanations of questions. Use your interpretation and understanding to answer the questions.
- You need not include all the columns(features) while answering questions, you can select a few meaningful columns and do analysis on them. Try to cover at least 50% of the columns(features) for the questions in this milestone.
- You MAY use a different kaggle notebook instead of the one shared with the host to do the task given in milestones.
- The answer length can vary depending on questions. Keep your answer concise and give a summary of the analysis and cover details using visualizations/bullet points wherever required. The maximum length of answer can be considered around 100 to 400 words.
- **Plots, graphs and figures:** add them in the "Figures" section at the end of this document, under the heading of the question they belong to, not inside the answer boxes. Number

each figure as Figure Q<question number>.<figure number> (for example, Figure Q3.1) and refer to it by this number in your answer (for example, "see Figure Q3.1").

# Q1. Model Building Plan

Identify suitable models for the given prediction problem and explain the planned model-building approach.

One suggested approach would be as follows:

- Identify suitable linear, non-tree, tree-based, and ensemble models for the prediction problem, where applicable.
- Justify the selection of models based on the nature of the target variable, dataset characteristics, and relevant modelling assumptions.

**Problem.** Predict price_tier (1 = cheapest … 4 = most expensive) for 114,538 test spaces from 59 features covering the operator, location, capacity, booking terms, availability, reviews and facilities. The target is a **4-class ordinal label** with fairly balanced classes (23.9% / 29.1% / 21.7% / 25.2% of 279,746 rows), and the metric is **macro-F1**, so every tier counts equally.

**Data characteristics that drive model choice**

• Mixed types: ~50 numeric and 7 categorical columns. Three are high-cardinality: zone (1,380 levels), space_class (131), borough (55). There is also a JSON list of 144 facility codes, plus dates.

• Structured missingness: 19.4% of rows have no reviews (all score_* columns missing together), and borough is 76.7% missing.

• Strong non-linear and interaction effects. Median max_seats is 2/2/4/5 by tier (Figure Q1.1), and 85% of shared_zone spaces are tier 1.

• Large n (280k): KNN and SVM get expensive, while tree ensembles scale well.

**Selected models**

• **Linear:** multinomial Logistic Regression.

• **Non-tree, non-linear:** K-Nearest Neighbours.

• **Tree:** Decision Tree.

• **Bagging:** Random Forest.

• **Boosting:** LightGBM and XGBoost.

• **Ensembles:** soft/hard Voting and Stacking.

**Plan.** Stateless feature engineering, then one sklearn Pipeline per model, with target encoding, scaling and imputation fitted inside each CV fold to avoid leakage. The engineered features lift a quick LightGBM from 0.7173 (raw columns) to 0.7321 macro-F1. Baselines are scored on a stratified 80/20 hold-out (55,950 rows). Tuning uses stratified 3-fold CV on a 30k sample, and before/after scores use the same hold-out.

**Baseline hold-out macro-F1** (Figure Q1.1): Random Forest 0.719, XGBoost 0.715, LightGBM 0.698, Logistic Regression 0.660, Decision Tree 0.614, KNN 0.568.

```
X_full, X_test = build_features(train_raw), build_features(test_raw)   # stateless feature
engineering
X_tr, X_val, y_tr, y_val = train_test_split(X_full, y_full, test_size=0.2, stratify=y_full,
random_state=42)

baseline_models = {
    'Logistic Regression': make_pipeline(make_linear_preprocessor(),
LogisticRegression(max_iter=1000)),
    'KNN (50k sample)':    make_pipeline(make_linear_preprocessor(), KNeighborsClassifier()),
    'Decision Tree':       make_pipeline(make_tree_preprocessor(),
DecisionTreeClassifier(random_state=42)),
    'Random Forest':       make_pipeline(make_tree_preprocessor(),
RandomForestClassifier(n_estimators=100)),
    'LightGBM':            make_pipeline(make_tree_preprocessor(impute=False), lgb.LGBMClassifier()),
    'XGBoost':             make_pipeline(make_tree_preprocessor(impute=False),
                                         EncodeLabels(xgb.XGBClassifier(tree_method='hist'))),
}
for name, model in baseline_models.items():
    X_fit, y_fit = (X_tr.loc[knn_idx], y_tr.loc[knn_idx]) if name.startswith('KNN') else (X_tr, y_tr)
    model.fit(X_fit, y_fit)
    print(name, f1_score(y_val, model.predict(X_val), average='macro'))
```

# Q2. Linear Models

Explore linear models and describe how their assumptions and characteristics influence their behavior on the given dataset. Implement and explore at least one suitable model, identify and describe the role of hyperparameters of the model.

One suggested approach would be as follows:

- Explain the fundamental assumptions of the selected linear model and how it represents the relationship between input features and the target variable.
- Discuss the impact of preprocessing steps on the model. Describe the strength and limitations of the model based on the dataset.
- Explain the role of relevant hyperparameters of the selected model and describe how they influence the model.

**Model.** Multinomial Logistic Regression models the log-odds of each tier as a linear function of the features, $z_k = w_k \cdot x + b_k$. Softmax turns these into probabilities, and training minimises cross-entropy plus an L2 penalty (lbfgs solver).

**Assumptions:** each feature has an additive, monotonic effect on the log-odds; classes are separable by hyperplanes; observations are independent; there is no severe multicollinearity.

**Behaviour on this data.** Baseline hold-out macro-F1 = **0.660**; tuned = **0.664**. That is far below the boosted trees (0.744), because the model cannot represent interactions such as "seats matter differently in shared vs whole units". The largest tier-4 coefficients are metro one-hots (MTR-08 +2.47, MTR-14 +1.06), zones, target-encoded zone and max_seats. The largest tier-1 coefficients are other metros and access_type = shared_zone (+0.57). The model is mostly learning *where* a space is and how big it is.

**Preprocessing impact** (3-fold CV, 30k sample)

- **No scaling: 0.417.** lbfgs struggles to converge, and the penalty hits large-range features unevenly.
- **Scaling: 0.641.**
- **Scaling + log1p** on 25 skewed counts: **0.643** (used).
- One-hot for low-cardinality categoricals, target encoding for zone/space_class/borough, and median imputation with missing indicators.

**Strengths:** fast (~32 s on 224k rows), interpretable, calibrated probabilities.

**Limitations:** linear boundaries only, and sensitive to scaling and outliers.

**Hyperparameters**

- **C** (inverse regularisation): CV rises from 0.613 at C = 0.001 to 0.643 at C = 10 and flattens above C = 1 (Figure Q2.1). Strong regularisation underfits here; the model is bias-limited, not variance-limited.
- **class_weight**: 'balanced' was selected.
- **penalty / solver** and **max_iter** (convergence).

```
def make_linear_preprocessor():
    log_scale = Pipeline([('impute', SimpleImputer(strategy='median')),
                          ('log', FunctionTransformer(np.log1p)), ('scale', StandardScaler())])
    plain = Pipeline([('impute', SimpleImputer(strategy='median', add_indicator=True)), ('scale',
StandardScaler())])
    return ColumnTransformer([
        ('target_enc', make_pipeline(TargetEncoder(random_state=42), StandardScaler()), HIGH_CARD),
        ('freq_enc', make_pipeline(FrequencyEncoder(), FunctionTransformer(np.log1p),
StandardScaler()), FREQ_COLS),
        ('onehot', OneHotEncoder(handle_unknown='ignore'), LOW_CARD),
        ('log_num', log_scale, SKEWED), ('num', plain, PLAIN)])

lr_pipe = make_pipeline(make_linear_preprocessor(), LogisticRegression(max_iter=2000))
lr_grid = {'logisticregression__C': [0.001, 0.01, 0.1, 1, 10],
           'logisticregression__class_weight': [None, 'balanced']}
lr_search = GridSearchCV(lr_pipe, lr_grid, scoring='f1_macro', cv=StratifiedKFold(3, shuffle=True,
random_state=42))
lr_search.fit(X_tune, y_tune)
# preprocessing ablation: no scaling / scaling without log / log + scaling
for label, pre in [('no scaling', no_scale), ('scaling, no log', no_log), ('log + scaling',
make_linear_preprocessor())]:
    print(label, cross_val_score(lr_variant(pre), X_tune, y_tune, scoring='f1_macro', cv=CV3).mean())
best_lr = clone(lr_pipe).set_params(**lr_search.best_params_).fit(X_tr, y_tr)
```

# Q3. Other Non-Tree Models

Build a suitable non-tree and non linear model for the given prediction problem. Choose at least one model and describe the underlying assumptions, mechanisms and behavior on the given dataset. Discuss the impact of preprocessing,  identify and describe the role of hyperparameters of the model.

One suggested approach would be as follows:

- Explain the fundamental assumptions of the selected model and how it represents the relationship between input features and the target variable.
- Discuss the impact of preprocessing steps on the model. Describe the strength and limitations of the model based on the dataset.
- Explain the role of relevant hyperparameters of the selected model and describe how they influence the model.

**Model.** K-Nearest Neighbours (non-parametric, non-linear). It labels a space with the majority tier of its **k** nearest training points (Euclidean distance on the scaled features), optionally weighting each neighbour by 1/distance.

**Assumption:** spaces that are close in feature space have similar tiers. There is no assumption about the shape of the decision boundary.

**Behaviour on this data.** Weakest model: baseline (k = 5) **0.568**, tuned **0.592** hold-out macro-F1. With ~240 encoded dimensions (134 binary facility flags), distances become less informative, and every feature counts equally in the distance, so weak features dilute strong ones like max_seats and location.

**Preprocessing impact**

- **Scaling is mandatory**; otherwise open_slots_d (0–365) dominates the distance.
- log1p limits the effect of outliers.
- Imputation is required.
- Target encoding keeps the dimensionality manageable.

**Training-set size:** 10k rows 0.537, 25k 0.572, 50k 0.592. Each extra row helps, but prediction time grows with it (1.9 s → 8.2 s for 56k predictions), so I kept 50k.

**Strengths:** no training, captures local structure.

**Limitations:** slow predictions, curse of dimensionality, no feature importance.

**Hyperparameters**

- **n_neighbors (k):** CV rises from 0.516 (k = 5) to a peak of **0.541 at k = 21**, then falls to 0.530 at k = 81 (Figure Q3.1). Small k is noisy (variance); large k blurs neighbouring tiers (bias).
- **weights:** distance beats uniform at every k (e.g. 0.541 vs 0.536 at k = 21).
- **metric:** Euclidean with brute-force search, which is fastest in high dimensions.

```
knn_pipe = make_pipeline(make_linear_preprocessor(), KNeighborsClassifier(algorithm='brute'))
knn_grid = {'kneighborsclassifier__n_neighbors': [5, 11, 21, 41, 81],
            'kneighborsclassifier__weights': ['uniform', 'distance']}
knn_search = GridSearchCV(knn_pipe, knn_grid, scoring='f1_macro', cv=CV3).fit(X_tune.iloc[:15000],
y_tune.iloc[:15000])

for n in [10000, 25000, 50000]:            # effect of training-set size
    idx = X_tr.sample(n, random_state=42).index
    m = clone(knn_pipe).set_params(**knn_search.best_params_).fit(X_tr.loc[idx], y_tr.loc[idx])
    print(n, f1_score(y_val, m.predict(X_val), average='macro'))
```

# Q4. Decision Trees

Explore decision trees and understand how they learn from data. Describe the underlying assumptions, mechanisms and behavior on the given dataset. Discuss the impact of preprocessing, identify and describe the role of hyperparameters of the model.

One suggested approach would be as follows:

- Explain the process of the decision tree model and how it represents the relationship between input features and the target variable.
- Discuss the impact of preprocessing steps on the model. Describe the strength and limitations of the model based on the dataset.
- Explain the role of relevant hyperparameters of the selected model and describe how they influence the model.

**Process.** A CART decision tree repeatedly picks the feature and threshold that most reduce impurity (Gini = $1 - \Sigma p_k^2$) and splits the node in two, until a stopping rule fires. Each leaf predicts its majority tier. The model is a set of if-then rules, so interactions are captured naturally. The top of the tuned tree (Figure Q4.2) splits on capacity, access type and target-encoded zone/metro, matching the EDA.

**Behaviour on this data.** The unrestricted tree scores **0.614** hold-out macro-F1; tuned (max_depth = 10, min_samples_leaf = 10) it scores **0.644**. Figure Q4.1 shows classic overfitting. With min_samples_leaf = 1, training macro-F1 climbs from 0.60 (depth 6) to 0.89 (depth 24+), while validation peaks at **depth 10 (0.592)** and then drops to 0.55. Past depth ~10 the tree memorises noise.

**Preprocessing impact**

- No scaling or log transforms: splits only use the ordering of values.
- Ordinal encoding is enough for low-cardinality categoricals.
- Target encoding lets one split separate cheap from expensive zones.
- Median imputation, with has_reviews kept as a flag.

**Strengths:** interpretable, handles non-linearity, fast (~11 s).

**Limitations:** high variance and greedy, axis-aligned splits, so it scores far below the ensembles (0.644 vs 0.725–0.744).

**Hyperparameters**

- **max_depth:** the main bias–variance control (best 10).
- **min_samples_leaf:** larger leaves make the score much less sensitive to depth. With leaf = 50, CV stays at ~0.588 for every depth ≥ 14, while leaf = 1 collapses to 0.55.
- **criterion:** gini vs entropy (gini chosen).
- **ccp_alpha:** post-pruning.

```
def make_tree_preprocessor(impute=True):
    return ColumnTransformer([
        ('target_enc', TargetEncoder(random_state=42), HIGH_CARD),
        ('freq_enc', FrequencyEncoder(), FREQ_COLS),
        ('ordinal', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1), LOW_CARD),
        ('num', SimpleImputer(strategy='median') if impute else 'passthrough', NUM_COLS)])

dt_pipe = make_pipeline(make_tree_preprocessor(), DecisionTreeClassifier(random_state=42))
dt_grid = {'decisiontreeclassifier__max_depth': [6, 10, 14, 18, 24, None],
           'decisiontreeclassifier__min_samples_leaf': [1, 10, 50, 200],
           'decisiontreeclassifier__criterion': ['gini', 'entropy']}
dt_search = GridSearchCV(dt_pipe, dt_grid, scoring='f1_macro', cv=CV3,
return_train_score=True).fit(X_tune, y_tune)
best_dt = clone(dt_pipe).set_params(**dt_search.best_params_).fit(X_tr, y_tr)
plot_tree(best_dt[-1], max_depth=2, feature_names=list(best_dt[0].get_feature_names_out()),
filled=True)
```

# Q5. Bagging Ensembles

Explore bagging-based ensemble models and understand how they learn from data. Describe the underlying mechanisms and behavior of these models on the given dataset. Discuss the impact of preprocessing, identify and describe the role of hyperparameters of the selected model.

One suggested approach would be as follows:

- Explain the working mechanism of the selected bagging model and how it combines multiple base models to make predictions.
- Discuss the impact of preprocessing steps on the model. Describe the strength and limitations of the model based on the dataset.
- Explain the role of relevant hyperparameters of the selected model and describe how they influence the model.

**Model.** Random Forest. It trains many deep trees, each on a **bootstrap sample** of the rows and with a **random subset of features** (max_features) considered at each split, then averages their class probabilities. De-correlated trees plus averaging keep the low bias of deep trees but cut their variance. The rows a tree never saw (**out-of-bag**, ~37%) give a free validation set.

**Behaviour on this data.** Baseline (100 trees) **0.719**, tuned **0.725** hold-out macro-F1, versus 0.614 for a single unpruned tree. That +0.105 jump shows the single tree's problem was variance. Out-of-bag macro-F1 on the 30k sample rises with the number of trees: 0.591 (10), 0.645 (25), 0.663 (50), 0.675 (100), 0.681 (200), flattening after ~100 (Figure Q5.1). It stays below boosting (0.744) because independent trees cannot correct their shared mistakes on the tier 2 vs 3 boundary. Top features by impurity importance are the target-encoded zone, max_seats, zones, workstations and the coordinates (Figure Q5.2).

**Preprocessing impact**

- Same as for trees: no scaling, ordinal plus target encoding.
- sklearn forests need median imputation.
- Correlated feature groups (open_slots_a…e) share their importance between them.

**Strengths:** robust and parallel, with an OOB estimate and importances.

**Limitations:** memory and time (tuned fit 137 s), less interpretable.

**Hyperparameters**

- **n_estimators:** more trees mean lower variance with diminishing returns.
- **max_features:** 0.35 was chosen, ahead of 'sqrt'. With ~200 features, sqrt (~14) made trees too weak.
- **max_depth / min_samples_leaf:** best None / 1, since deep trees are fine once averaged.
- **class_weight:** 'balanced_subsample' was chosen.

```
rf_pipe = make_pipeline(make_tree_preprocessor(), RandomForestClassifier(random_state=42))
rf_space = {'randomforestclassifier__n_estimators': [100, 200],
            'randomforestclassifier__max_features': ['sqrt', 0.2, 0.35],
            'randomforestclassifier__max_depth': [None, 20, 30],
            'randomforestclassifier__min_samples_leaf': [1, 2, 5],
            'randomforestclassifier__class_weight': [None, 'balanced_subsample']}
rf_search = RandomizedSearchCV(rf_pipe, rf_space, n_iter=6, scoring='f1_macro', cv=CV3,
random_state=42)
rf_search.fit(X_tune, y_tune)

for n in [10, 25, 50, 100, 200]:              # out-of-bag score vs number of trees
    rf_ = RandomForestClassifier(**dict(best_rf_params, n_estimators=n), oob_score=True).fit(Xt,
y_tune)
    oob_pred = rf_.classes_[rf_.oob_decision_function_.argmax(1)]
    print(n, f1_score(y_tune, oob_pred, average='macro'))
```

# Q6. Boosting Ensembles

Explore boosting-based ensemble models and understand how they learn from data. Describe the underlying mechanisms and behavior of these models on the given dataset. Discuss the impact of preprocessing, identify and describe the role of hyperparameters of the selected model.

One suggested approach would be as follows:

- Explain the working mechanism of the selected boosting model and how it is built.
- Discuss the impact of preprocessing steps on the model. Describe the strength and limitations of the model based on the dataset.
- Explain the role of relevant hyperparameters of the selected model and describe how they influence the model.

**Model.** LightGBM (gradient-boosted trees), with XGBoost as a second implementation. Boosting adds trees **sequentially**. Each round fits small trees (one per class) to the **negative gradient of the multi-class log-loss**, i.e. what the ensemble still gets wrong, and adds them with shrinkage: $F_m = F_{m-1} + \eta \cdot tree_m$. Every tree corrects the previous ones, so boosting reduces bias as well as variance. LightGBM uses histogram binning, leaf-wise growth and native NaN handling.

**Behaviour on this data.** Default LightGBM (100 trees, 31 leaves) gives only **0.698**: too few trees for 280k rows. Tuned, with learning rate 0.05 and **2,078 trees** chosen by early stopping, it reaches **0.744**, the best single model. Tuned XGBoost gives 0.742. Boosting beats Random Forest (0.725) because sequential correction focuses on the hard tier 2/3 boundary.

**Preprocessing impact**

- No scaling. NaNs are passed through, so no imputation is needed.
- In an exploratory check, target encoding and LightGBM's native categorical handling scored the same (0.7404 vs 0.7403), so I kept the sklearn pipeline.
- Feature engineering adds +0.015 (0.717 → 0.732, quick LightGBM).

**Strengths:** highest accuracy, fast, handles NaN and mixed types.

**Limitations:** many interacting hyperparameters, needs early stopping, less interpretable.

**Hyperparameters**

- **learning_rate × n_estimators:** at lr = 0.1, early stopping picks 978 trees (early-stopping-set macro-F1 0.737). At lr = 0.05 it picks 2,078 trees (0.743), i.e. smaller steps need about twice the trees but generalise slightly better (Figure Q6.1).
- **num_leaves:** 63 best.
- **min_child_samples:** 10.
- **subsample / colsample_bytree:** 0.85 / 0.5.
- **reg_lambda:** 0.

```
lgb_pipe = make_pipeline(make_tree_preprocessor(impute=False), lgb.LGBMClassifier(subsample_freq=1,
verbose=-1))
lgb_space = {'lgbmclassifier__n_estimators': [200, 400], 'lgbmclassifier__learning_rate': [0.03,
0.06, 0.1],
             'lgbmclassifier__num_leaves': [31, 63, 127, 255], 'lgbmclassifier__min_child_samples':
[10, 30, 80],
             'lgbmclassifier__subsample': [0.7, 0.85, 1.0], 'lgbmclassifier__colsample_bytree': [0.3,
0.5, 0.8],
             'lgbmclassifier__reg_lambda': [0, 1, 5]}
lgb_search = RandomizedSearchCV(lgb_pipe, lgb_space, n_iter=8, scoring='f1_macro', cv=CV3,
random_state=42)
lgb_search.fit(X_tune, y_tune)

# early stopping on a 10% split of the TRAINING part chooses the number of trees per learning rate
X_in, X_es, y_in, y_es = train_test_split(X_tr, y_tr, test_size=0.1, stratify=y_tr, random_state=42)
for lr in [0.1, 0.05]:
    m = lgb.LGBMClassifier(**dict(best_lgb_params, learning_rate=lr, n_estimators=3000))
    m.fit(Xin_t, y_in, eval_set=[(Xes_t, y_es)], callbacks=[lgb.early_stopping(100)])
```

# Q7. Voting and Stacking

Explore voting and stacking ensemble models and understand how the algorithm works. Describe the underlying mechanisms and behavior of these models on the given dataset. Discuss the impact of preprocessing, identify and describe the role of hyperparameters of the selected model.

One suggested approach would be as follows:

- Explain the working mechanisms of voting and stacking models and how they combine predictions from multiple base models to make final predictions.
- Discuss the impact of preprocessing steps on the model. Describe the strength and limitations of the model based on the dataset.
- Explain the role of relevant hyperparameters of the selected model and describe how they influence the model.

**Voting.** Several independently trained models are combined:

- **Hard voting:** the majority label wins.
- **Soft voting:** a (weighted) average of the class probabilities.

**Stacking.** A **meta-learner** (logistic regression) is trained on the base models' **out-of-fold** probabilities (2-fold internal CV, so no row is scored by a model that saw it). It learns how much to trust each model per tier.

**Setup.** The tuned base models come from different families: Logistic Regression (0.664), Random Forest (0.725), LightGBM (0.744) and XGBoost (0.742; voting only). Each keeps its **own preprocessing pipeline** inside the ensemble.

**Results** (hold-out macro-F1, Figure Q7.1)

- Hard voting: 0.739
- Soft voting, equal weights: 0.743 (the weak LR pulls it down)
- Soft voting, boosting ×2: 0.744
- Soft voting without LR: 0.745
- **Stacking: 0.7456, the best model overall**, only +0.0014 over tuned LightGBM.

The stacking meta-learner's total |coefficient| per base model is LightGBM 29.3, RF 7.1, LR 3.2, so it learned to rely almost entirely on boosting.

**Interpretation.** The gain is small because the strong base models make highly correlated errors (mostly tier 2 vs 3). There is little left for an ensemble to fix.

**Strengths:** best score, and stacking learns the per-model trust itself.

**Limitations:** cost. Stacking took 1,022 s versus 274 s for LightGBM alone, and the final refit took 1,299 s. It is also hard to interpret.

**Hyperparameters**

- Voting: voting (soft/hard), weights, and which estimators to include.
- Stacking: final_estimator, cv, stack_method (predict_proba), passthrough.

```
vote = VotingClassifier([('lr', best_lr), ('rf', best_rf), ('lgb', lgb_es), ('xgb', best_xgb)],
voting='soft')
vote.fit(X_tr, y_tr)
for params in [dict(voting='soft', weights=None), dict(voting='soft', weights=[1, 1, 2, 2]),
               dict(voting='soft', weights=[0, 1, 1, 1]), dict(voting='hard', weights=None)]:
    vote.set_params(**params)                     # used only at predict time: no refit needed
    print(params, f1_score(y_val, vote.predict(X_val), average='macro'))

stack = StackingClassifier(estimators=[('lr', best_lr), ('rf', best_rf), ('lgb', lgb_es)],
                           final_estimator=LogisticRegression(max_iter=2000),
                           cv=StratifiedKFold(2, shuffle=True, random_state=42),
stack_method='predict_proba')
stack.fit(X_tr, y_tr)
print('stacking', f1_score(y_val, stack.predict(X_val), average='macro'))
```

# Q8. Hyperparameter Tuning(HPT) Strategy

Explain the process of selecting suitable hyperparameter configurations for machine learning models. Explore how cross-validation is used to obtain reliable estimates of model performance and how different search strategies can be used to identify suitable configurations.

One suggested approach would be as follows:

- Explain the factors that should be considered when selecting hyperparameters and defining their search ranges. Discuss how the choice of hyperparameter values can influence the trade-off between model complexity and generalisation.
- Explore different cross-validation strategies and describe the importance and implementation steps of them.
- Explore HPT approaches. Explain how they explore the search space and compare their advantages, limitations, and computational requirements.

**Selecting hyperparameters and ranges**

• Tune the parameters that control **model complexity** (bias–variance): C, k, max_depth/min_samples_leaf, num_leaves/learning_rate/n_estimators, max_features.

• Use log scales for C and the learning rate.

• Include both an underfitting and an overfitting end, so the optimum falls inside the range. For example, the decision tree's validation score peaks at depth 10 and falls at both ends (Figure Q4.1), and LR's C curve rises then flattens (Figure Q2.1).

• Data facts guide the ranges: balanced classes make class_weight minor, and ~240 dimensions argue for a larger k.

**Cross-validation strategies** (tuned LR, 30k sample, Figure Q8.1)

• Repeated 80/20 hold-out (5 seeds): 0.645 ± 0.004

• KFold-5 (not stratified): 0.644 ± 0.004

• **StratifiedKFold-3: 0.643 ± 0.002**

• StratifiedKFold-5: 0.644 ± 0.004

• StratifiedKFold-10: 0.645 ± 0.005

All means agree within 0.002, so with 30k rows the strategy hardly changes the estimate. Stratified 3-fold was the cheapest (3.7 s vs 48 s for repeated hold-out) with a low spread, so I used it for all searches. Stratification guarantees every fold keeps the 24/29/22/25% tier mix. All learned preprocessing sits inside the Pipeline, so it is refitted on training folds only.

**Search approaches** (decision tree, same 48-configuration space)

• **Grid:** 48 configs, 33.3 s, best 0.5921 (depth 10, leaf 10).

• **Random:** 12 configs, 8.4 s, **0.5920**, i.e. the same optimum at a quarter of the cost.

• **Successive halving:** 48 candidates, 8.6 s, 0.5711. Its early rounds on 2k rows favoured large leaves (leaf 50), which suit small data, so it missed the best configuration.

• **Bayesian optimisation** (Optuna) is not on the allowed-library list.

**Conclusion:** grid search for small spaces, random search for large ones. Halving is fast but can be misled by small-sample rounds.

```
for label, cv in [('KFold k=5', KFold(5, shuffle=True, random_state=42)),
                  ('StratifiedKFold k=3', StratifiedKFold(3, shuffle=True, random_state=42)),
                  ('StratifiedKFold k=5', StratifiedKFold(5, shuffle=True, random_state=42)),
                  ('StratifiedKFold k=10', StratifiedKFold(10, shuffle=True, random_state=42))]:
    s = cross_val_score(best_lr, X_tune, y_tune, scoring='f1_macro', cv=cv)
    print(label, s.mean(), s.std())
# plus 5 repeated 80/20 hold-out splits for comparison

grid = GridSearchCV(dt_pipe, dt_grid, scoring='f1_macro', cv=CV3).fit(X_tune, y_tune)               #
48 configs
rand = RandomizedSearchCV(dt_pipe, dt_grid, n_iter=12, scoring='f1_macro', cv=CV3).fit(X_tune,
y_tune)
halv = HalvingRandomSearchCV(dt_pipe, dt_grid, n_candidates=48, factor=3, min_resources=2000,
                             scoring='f1_macro', cv=CV3).fit(X_tune, y_tune)
```

# Q9. Hyperparameter tuning Effects

Perform hyperparameter tuning on at least three machine learning models to identify suitable hyperparameter configurations for the given prediction problem. Apply appropriate cross-validation and search strategies to evaluate different configurations.

One suggested approach would be as follows:

- Identify the important hyperparameters of models and define suitable ranges or candidate values for tuning. Explain the reasoning behind the selected search spaces.
- Choose an appropriate hyperparameter search strategy. Justify the choice based on the search space and computational requirements.
- Select an appropriate cross-validation strategy for each model and use it to evaluate different hyperparameter configurations.
- Report the best hyperparameter configuration identified for each model along with its corresponding cross-validation score. Describe the observations from the tuning process.

**Setup.** Every search optimises f1_macro with **StratifiedKFold(3, shuffle=True)** on a stratified 30,000-row sample of the training part (15k for KNN). The best configuration is then refitted on all 223,796 training rows and scored on the 55,950-row hold-out. Preprocessing sits inside each pipeline, so it is refitted per fold.

| Model | Search | Search space | Best configuration | CV macro-F1 |
|---|---|---|---|---|
| Logistic Regression | Grid, 10 configs | C ∈ {0.001 … 10} (log); class_weight ∈ {None, balanced} | C = 10, class_weight = balanced | 0.643 |
| KNN | Grid, 10 configs | k ∈ {5, 11, 21, 41, 81}; weights ∈ {uniform, distance} | k = 21, distance | 0.541 |
| Decision Tree | Grid, 48 configs | max_depth {6 … 24, None}; min_samples_leaf {1, 10, 50, 200}; criterion | depth 10, leaf 10, gini | 0.592 |
| Random Forest | Random, 6 of 108 | n_estimators {100, 200}; max_features {sqrt, 0.2, 0.35}; max_depth {None, 20, 30}; leaf {1, 2, 5}; class_weight | 200 trees, max_features 0.35, depth None, leaf 1, balanced_subsample | 0.659 |
| LightGBM | Random, 8 of 1,944 + early stopping | lr {0.03, 0.06, 0.1}; num_leaves {31 … 255}; min_child_samples; subsample; colsample; reg_lambda | 63 leaves, lr 0.1, min_child 10, subsample 0.85, colsample 0.5, λ = 0 → refit lr 0.05, 2,078 trees | 0.675 |
| XGBoost | Random, 4 of 96 | n_estimators; lr; max_depth {6, 8, 10}; min_child_weight; subsample; colsample | 600 trees, lr 0.1, depth 10, min_child_weight 5, subsample 0.8, colsample 0.7 | 0.675 |

**Why these strategies.** Grid search for the cheap models with 2–3 parameters. Random search for RF/LightGBM/XGBoost, whose spaces are too large and slow to enumerate on a 4-core CPU. For LightGBM, early stopping on a 10% split of the training part then picked the number of trees.

**Observations**

- Low regularisation won for LR (C = 10).
- KNN wanted a medium k (21) with distance weighting.
- The tree wanted depth 10, much shallower than unlimited.
- RF preferred more features per split (0.35).
- In LightGBM the top two configurations both had 63 leaves, lr 0.1 and colsample 0.5 (0.6750/0.6749). lr 0.03 with colsample 0.8 was worst (0.668). The whole random search spans only 0.668–0.675 (Figure Q9.1): LightGBM is robust, and the number of trees mattered more than the other parameters.
- CV scores are lower than the hold-out scores because tuning used only 30k rows.

```
X_tune, _, y_tune, _ = train_test_split(X_tr, y_tr, train_size=30000, stratify=y_tr, random_state=42)
CV3 = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
searches = {'Logistic Regression': lr_search,   # GridSearchCV, 10 configs
            'KNN': knn_search,                  # GridSearchCV, 10 configs (15k rows)
            'Decision Tree': dt_search,         # GridSearchCV, 48 configs
            'Random Forest': rf_search,         # RandomizedSearchCV, 6 of 108 configs
            'LightGBM': lgb_search,             # RandomizedSearchCV, 8 of 1,944 configs + early
stopping
            'XGBoost': xgb_search}              # RandomizedSearchCV, 4 of 96 configs
for name, s in searches.items():
    print(name, s.best_params_, round(s.best_score_, 4))
```

# Q10. Model Score Analysis

Compare the scores obtained from the initial models and the models after hyperparameter tuning.

One suggested approach would be as follows:

- Present the scores before and after hyperparameter tuning for the selected models.
- Identify which models showed improvement after tuning and describe the observed changes.

**Hold-out macro-F1 before and after tuning** (Figure Q10.1)

| Model | Before tuning | After tuning | Change |
|---|---|---|---|
| KNN (50k sample) | 0.568 | 0.592 | +0.025 |
| Decision Tree | 0.614 | 0.644 | +0.031 |
| Logistic Regression | 0.660 | 0.664 | +0.004 |
| Random Forest | 0.719 | 0.725 | +0.006 |
| XGBoost | 0.715 | 0.742 | +0.027 |
| LightGBM | 0.698 | 0.744 | +0.046 |

**Observations**

• **Every model improved.** The biggest gain was **LightGBM (+0.046)**: its default of 100 trees is far too few for 224k rows, and early stopping chose 2,078 trees at lr 0.05.

• **Decision Tree (+0.031) and KNN (+0.025)** gained because tuning fixed their variance (depth limit; larger k with distance weighting).

• **XGBoost (+0.027)** benefited from deeper trees (depth 10) and more rounds (600).

• **Logistic Regression (+0.004) and Random Forest (+0.006)** barely moved. LR is limited by linearity, not regularisation, and RF is already robust at defaults.

• The ranking changed: before tuning RF was best (0.719). After tuning, the boosting models lead (0.744/0.742) and **stacking is best overall at 0.7456** (0.748 accuracy).

• **Best model per tier** (stacking): F1 = 0.842 (tier 1), 0.716 (tier 2), 0.606 (tier 3), 0.819 (tier 4). The extreme tiers are easy. Most errors are between adjacent middle tiers 2 and 3 (Figure Q10.2), which overlap in price.

• The final stacking model, refitted on all 279,746 rows, predicts a test distribution of 22.6% / 33.3% / 20.3% / 23.8%, close to the training mix. Its hold-out score is well above the 0.70 cut-off.

```
res = pd.DataFrame(RESULTS)                          # every evaluate() call logs model, stage, hold-
out macro-F1
ba = res[res.stage.isin(['baseline', 'tuned'])]
before_after = ba.pivot_table(index='model', columns='stage', values='holdout_macro_f1',
aggfunc='max')
before_after['gain'] = before_after['tuned'] - before_after['baseline']
print(before_after.sort_values('tuned'))
print(classification_report(y_val, best_model.predict(X_val), digits=4))
```

# Figures

Add all plots, graphs and figures for this milestone in this section, under the heading of the question they belong to.

**How to add a figure**

1. Place your cursor below the heading of the question the figure belongs to, and use Insert > Image to add the figure.
2. Number each figure in the format Figure Q<question number>.<figure number>. For example, the first two figures for Q3 are Figure Q3.1 and Figure Q3.2.
3. Write the figure number and a short title on the line just above the figure, for example: Figure Q9.1: Validation score for different values of a hyperparameter.
4. In your answer, refer to the figure by its number, for example: "The score stops improving beyond a certain value of the hyperparameter (see Figure Q9.1)."
5. Resize each figure so that it fits within the page width and all axis labels and legends are readable.
6. Replace the placeholder lines under each heading with your figures. If a question has no figures, leave its heading as it is.

## Figures for Q1. Model Building Plan

*Figure Q1.1: Baseline models: hold-out macro-F1*

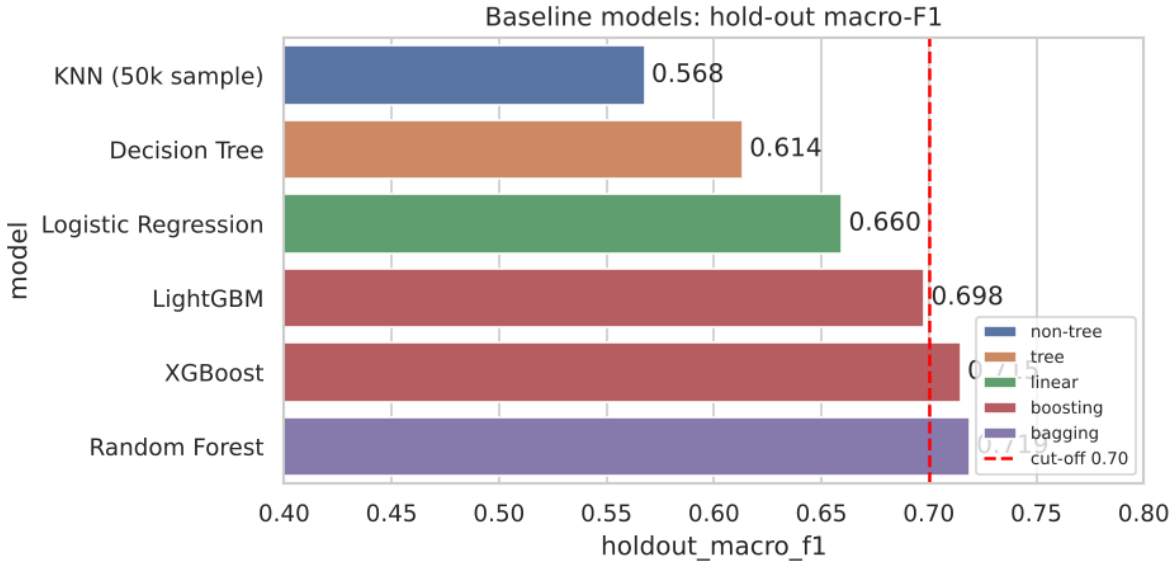
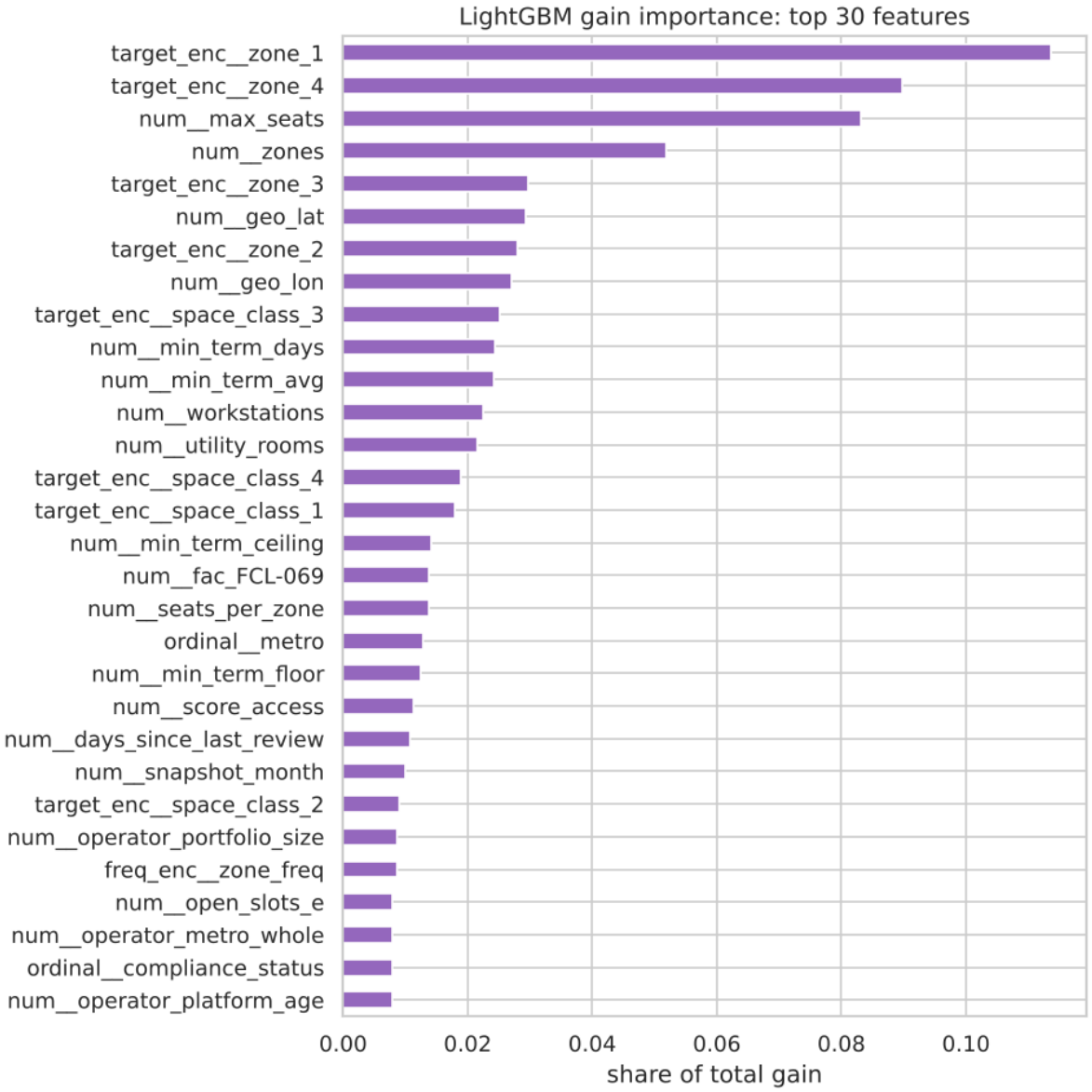
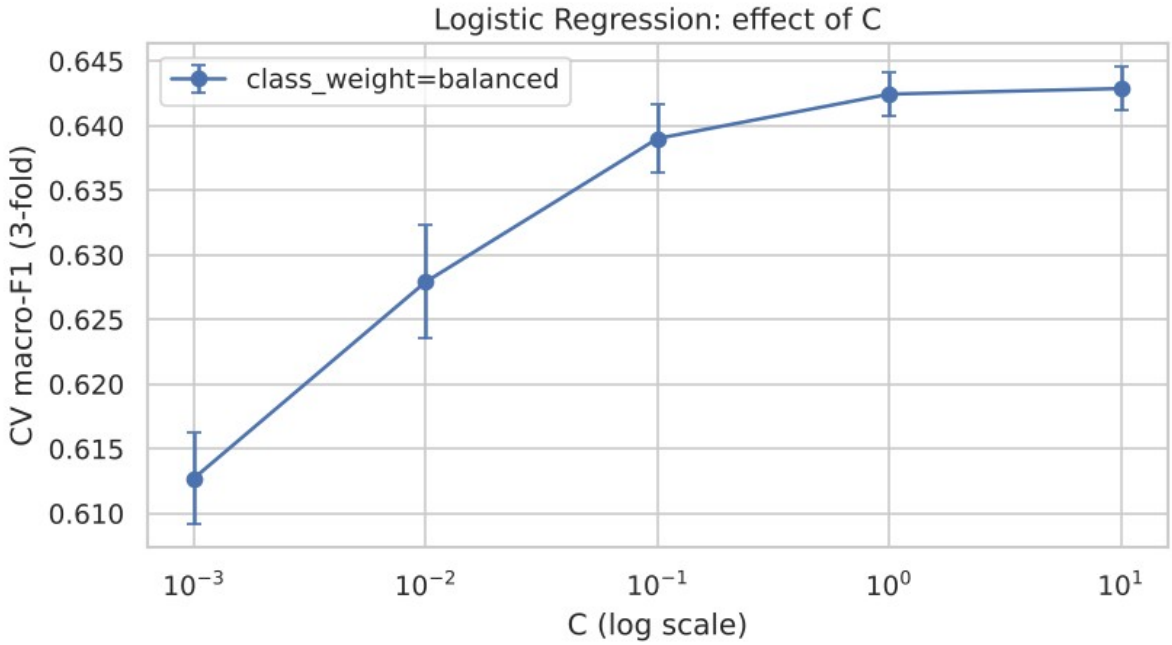


*Figure Q1.2: LightGBM gain importance with engineered features (top 30)*



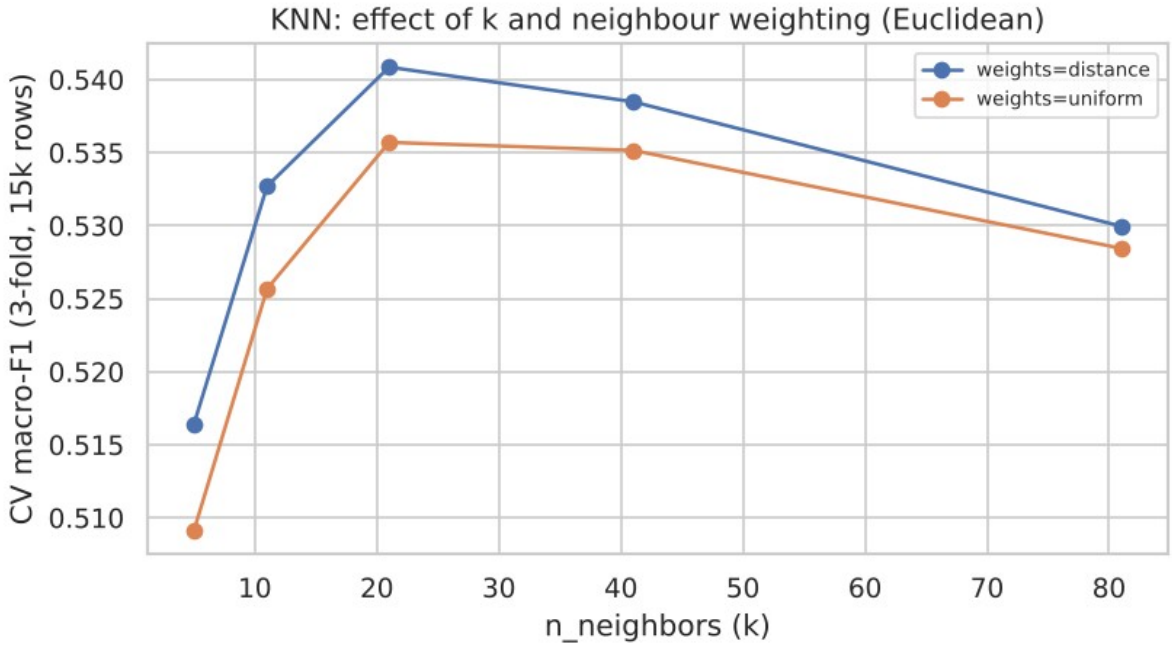
## Figures for Q2. Linear Models

*Figure Q2.1: Logistic regression: 3-fold CV macro-F1 vs C (class_weight = balanced)*




## Figures for Q3. Other Non-Tree Models

*Figure Q3.1: KNN: CV macro-F1 vs k for uniform and distance weighting*



## Figures for Q4. Decision Trees

*Figure Q4.1: Decision tree: CV macro-F1 vs depth and leaf size, and train vs validation*

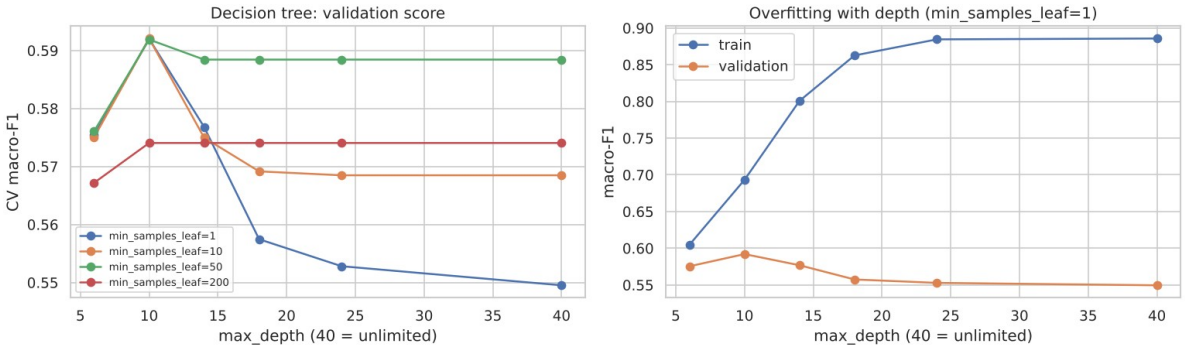
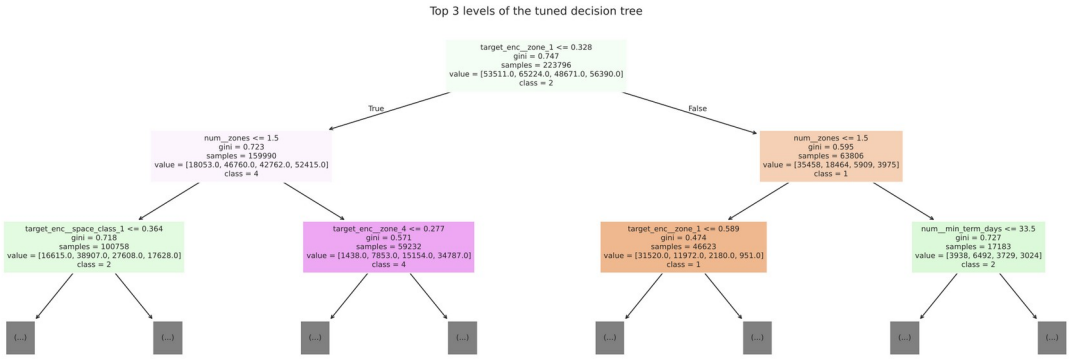



*Figure Q4.2: Top three levels of the tuned decision tree*



## Figures for Q5. Bagging Ensembles

*Figure Q5.1: Random forest: out-of-bag macro-F1 vs number of trees*

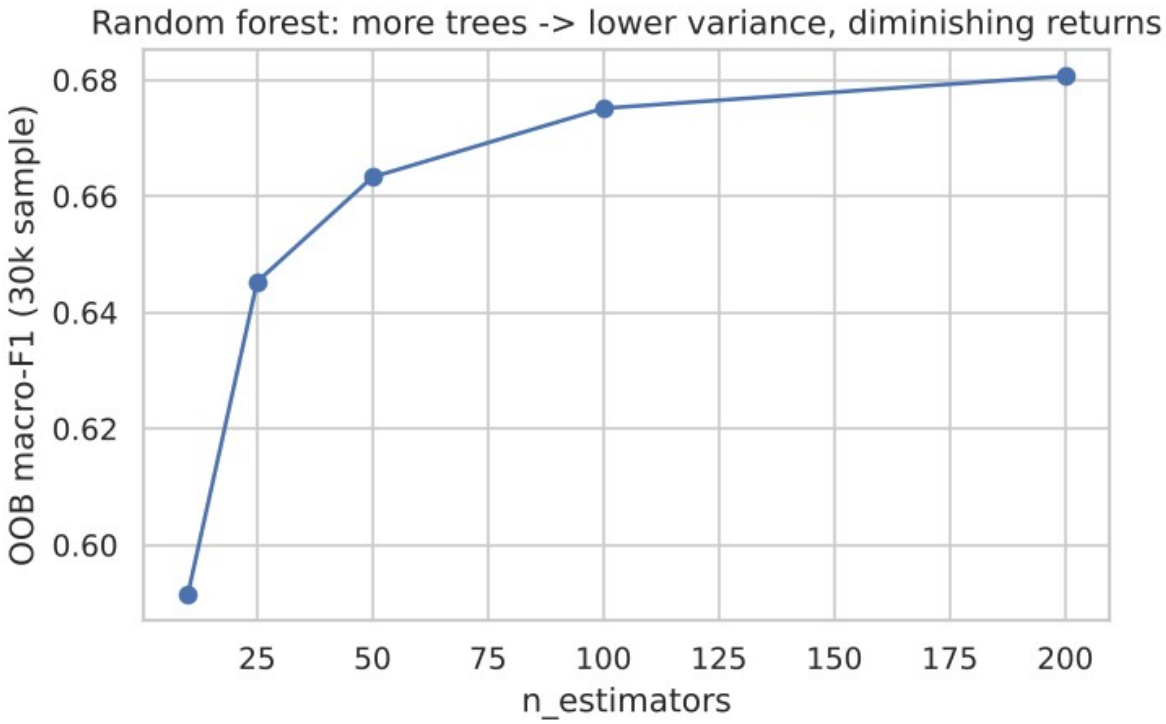
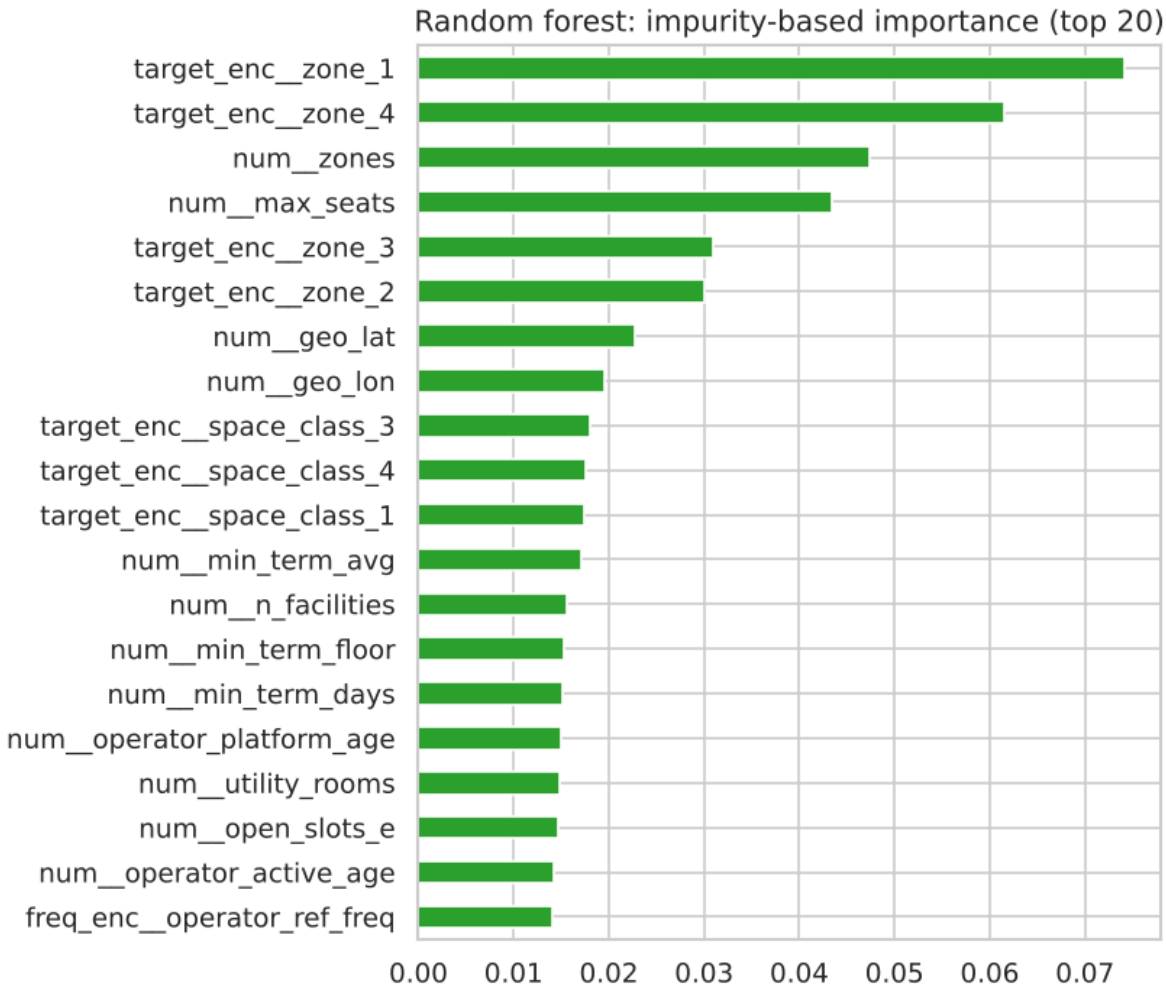
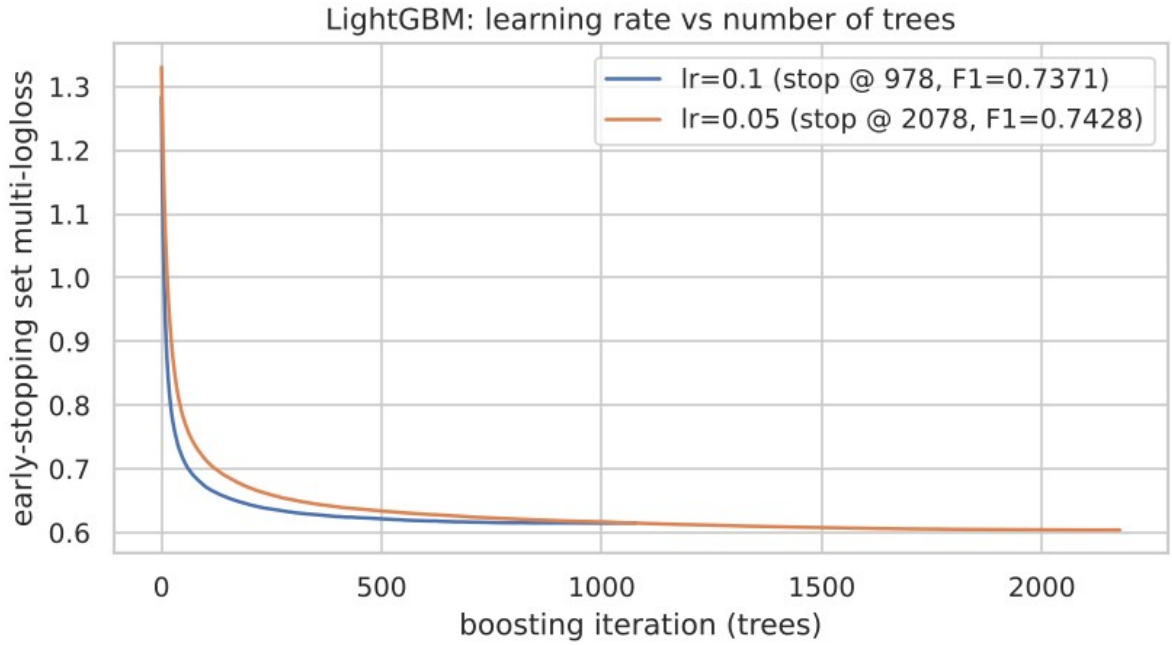


*Figure Q5.2: Random forest: top-20 impurity importances*



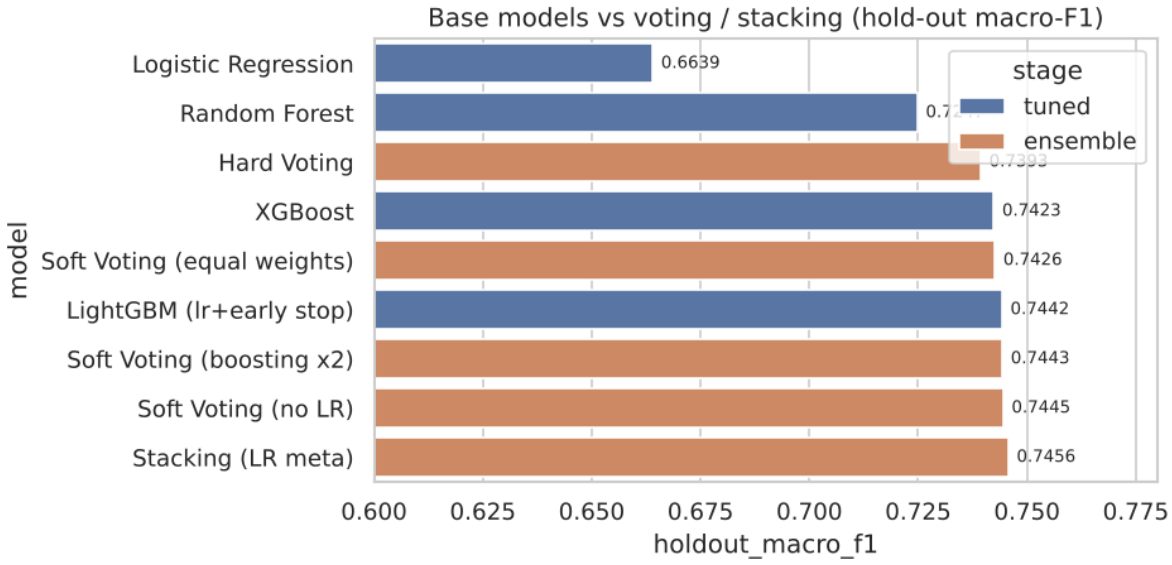
## Figures for Q6. Boosting Ensembles

*Figure Q6.1: LightGBM: early-stopping log-loss vs trees for learning rates 0.1 and 0.05*



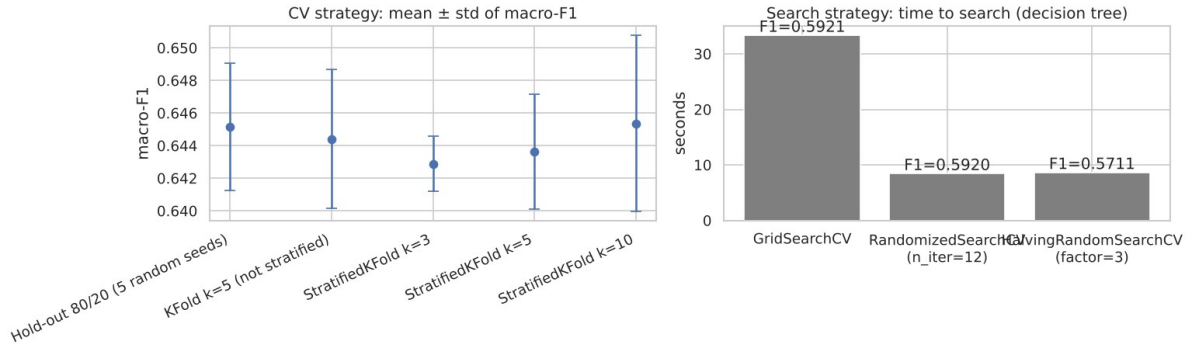

## Figures for Q7. Voting and Stacking

*Figure Q7.1: Tuned base models vs voting and stacking (hold-out macro-F1)*



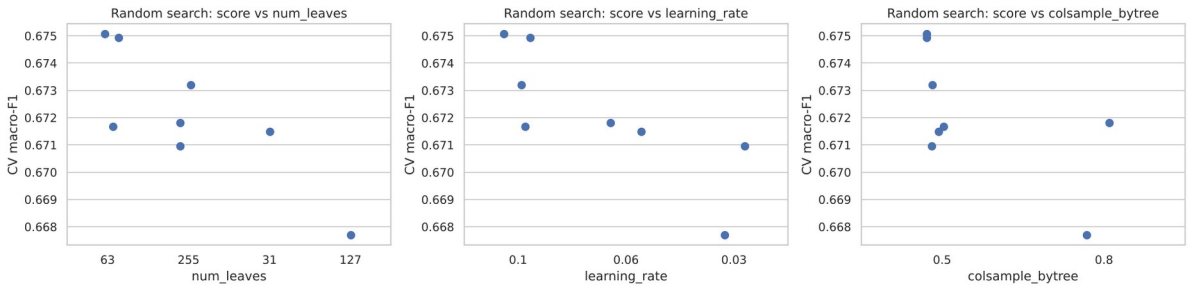
## Figures for Q8. Hyperparameter Tuning Strategy

*Figure Q8.1: Cross-validation strategies (mean ± std) and search-strategy time*




## Figures for Q9. Hyperparameter Effects and Configuration Rationale

*Figure Q9.1: LightGBM random search: CV macro-F1 per configuration*




## Figures for Q10. Model Score Analysis

*Figure Q10.1: Hold-out macro-F1 before vs after tuning*

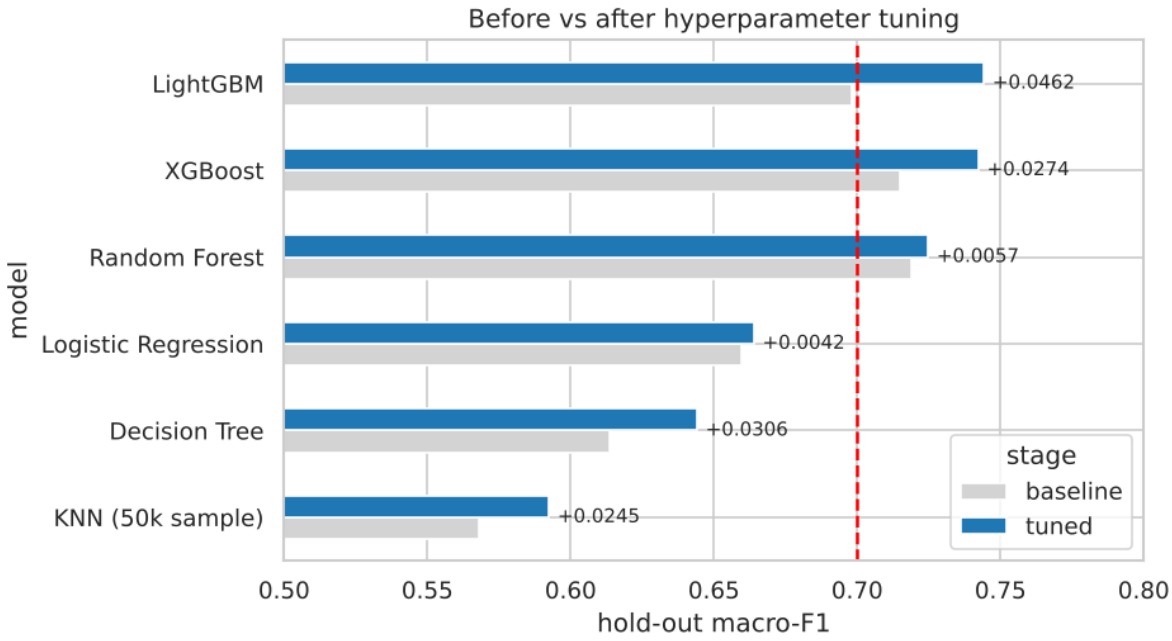


*Figure Q10.2: Confusion matrix of the stacking model on the hold-out*

